

Clahan Academy — Full Presentation Script

Cloud-Native Microservices Platform & GitOps CI/CD Lifecycle

Estimated Duration: 30–35 minutes

SLIDE 1 — Title Slide (Self Introduction)

[Stand confidently, make eye contact with the audience before you speak]

"Good [morning/afternoon], everyone.

My name is [Your Name], and I'm really glad to be here presenting today.

Before I dive in, I want to mention something I'm genuinely proud of — during this training period, I completed my **GH200 certification**. The GH200 is NVIDIA's Grace Hopper Superchip architecture, and working toward that certificate gave me a much deeper understanding of GPU-accelerated workloads — which, as you'll soon see, ties very directly into what we built here.

Today, I'll be walking you through **Clahan Academy** — a full-stack, cloud-native microservices platform built for online exam management and AI-powered proctoring. We'll go from the application layer all the way up through the cloud infrastructure, covering the *why*, the *what*, and the *how* at every step.

Let's get started."

[Pause, advance slide]

SLIDE 2 — Why Clahan Academy

[Speak with conviction — this is where you establish the problem and the purpose]

"Every good project starts with a question: **Why does this need to exist?**

The traditional approach to online exams is broken. Institutions rely on either honor-based systems — which are easy to cheat — or expensive third-party proctoring tools that are opaque, costly, and often invasive to students.

Clahan Academy was built to solve exactly that. The vision was a **self-hosted, fully controlled exam platform** that an organization can own end-to-end — from the student's login experience, all the way to real-time AI-powered proctoring and automated grading.

The key goals were:

- **Security** — so that exam integrity is never in question

- **Scalability** — so the system can handle hundreds of concurrent students without breaking a sweat
- **Automation** — so that grading, notifications, and audit trails happen without manual intervention
- **Cloud-nativity** — so the entire platform can be deployed, updated, and rolled back with confidence

This isn't just an academic exercise. Every decision we made — in the architecture, the infrastructure, the security model — was driven by real-world production thinking.

Let me now show you what we actually built."

[Pause, advance slide]

🔪 SLIDE 3 — (Visual / Transition Slide)

[If this is a diagram or overview visual, describe it briefly]

"Here's a high-level view of the platform before we zoom in. You can see it naturally separates into two worlds — the **application layer**, which is where all student and admin interactions happen, and the **cloud infrastructure layer**, which is what makes it all reliable, secure, and scalable.

We'll walk through both in detail."

[Advance slide]

🔪 SLIDE 4 — Application Architecture

"Let's start at the application level.

Clahan Academy follows a **microservices architecture**. Now, you've probably heard that term a lot — but the reason we chose it here is very intentional.

In a monolithic system, if the proctoring module has a bug or needs to scale, you'd have to redeploy the entire application. That's slow and risky. With microservices, each concern is isolated. We can **scale the AI service independently** when there's a heavy exam load, while keeping the notification service at minimal resources.

Each service has a single responsibility, communicates over well-defined APIs, and can be deployed, updated, or replaced without touching anything else.

Let me walk you through the specific services we built."

[Advance slide]

🔪 SLIDE 5 — Microservices Catalog Map

"Here's our full microservices catalog.

Starting from the front — students and admins interact through a **Vite and React frontend**. We chose Vite for its blazing fast build times and React for its component-based UI, which made building complex flows like the exam interface and proctoring client much more manageable.

Behind the frontend, the **Auth Service** handles everything related to identity — JWT-based authentication, session validation, and hooks for two-factor authentication. Security starts here.

Then we have three core business logic services:

- The **Student Service** manages student profiles and records
- The **Admin Service** handles exam creation, configuration, and management
- The **Exam Service** orchestrates the actual exam-taking flow — it's the brain of the operation

Now, this is where it gets interesting. The **Proctoring Service** is a WebSocket handler. It maintains a persistent, real-time connection with the student's browser and ingests webcam frames continuously during the exam.

Those frames are passed to the **AI Service** — which runs on GPU nodes — using **YOLOv8** for object detection and **InsightFace** for face recognition. This is where my GH200 certification knowledge really comes into play. Understanding how GPU memory, tensor cores, and inference pipelines work helped us think clearly about how to size and configure the AI workload.

Finally, the **Notification Service** is a background worker using **BullMQ** — a Redis-backed message queue — that handles sending email notifications via SMTP. This is asynchronous by design, so grading results don't wait on email delivery.

Every service has a job. None of them overstep it."

[Advance slide]

🔪 SLIDE 6 — Student Test-Taking Journey (Step-by-Step)

"Now that you know the services, let me show you how they all work together from a student's perspective. Let's follow a student through an entire exam.

Step 1 — Login. The student authenticates through the Auth Service. A JWT token is issued, the session is recorded, and from this point on, every request is validated against that token. No anonymous access anywhere.

Step 2 — Start Exam. The student clicks 'Start Exam.' Two things happen simultaneously: the Exam Service activates the exam session, and a **persistent WebSocket connection** is established with the Proctoring Service. The camera stream begins immediately.

Step 3 — Code Work. For programming questions, students write code directly inside the exam UI. Submissions are forwarded to **Judge0** — an open-source code execution sandbox — for compilation and execution in a completely isolated environment. We'll look at this in more detail shortly.

Step 4 — Submit Answers. When the student is done, all responses — both code and descriptive answers — are submitted to the Exam Service in one consolidated payload.

Step 5 — Automated Grading. This is one of the most powerful features. Descriptive answers are evaluated by **Ollama running the phi3 model** — a local LLM inference engine. This means AI-powered grading happens entirely on our infrastructure, with no data leaving our environment. Code is scored based on Judge0 execution results.

Step 6 — Notification. Once grading is complete, BullMQ enqueues a job. The Notification Service picks it up, emails the student their grade report, and creates an audit record. The exam lifecycle is complete.

The beautiful thing about this flow is that it's **fully automated**. An instructor sets up the exam. After that, everything — proctoring, grading, notification, audit — happens without any human intervention."

[Advance slide]

SLIDE 7 — Real-Time AI Proctoring — How It Works

"Let me zoom into the proctoring system because this is genuinely one of the most technically interesting parts of the project.

The flow looks like this:

First, the student's **browser opens a WebSocket** to the Proctoring Service. WebSockets are different from regular HTTP requests — they maintain a live, bidirectional connection. This is critical because we're not just sending occasional pings; we're streaming webcam frames continuously throughout the exam.

The Proctoring Service **receives those frames** and forwards them to the AI Service. The AI Service is where the real work happens — it runs **YOLOv8**, which is a state-of-the-art object detection model, to look for prohibited items in the frame. And it runs **InsightFace** for facial recognition — verifying that the person in the frame is actually the registered student, and that they haven't been replaced by someone else.

All of this runs on **GPU nodes in our Kubernetes cluster**. When an anomaly is detected — maybe a phone in the frame, maybe a second face — the AI Service **emits an alert immediately**. The proctor or admin can see this in real-time.

This isn't a black-box third-party tool. We own the model, the inference pipeline, the alerting logic. Everything is auditable, controllable, and on our infrastructure."

[Advance slide]

🔪 SLIDE 8 — Sandboxed Programming Workspace

"Let's talk about code execution, because running untrusted student code is a significant security challenge.

You cannot just let student-submitted code run on your servers. If someone submits a `while True: pass` loop, or worse, something malicious — you need to contain it completely.

Our solution is **Judge0** — an open-source judge API that wraps each code submission in a fully isolated container with strict resource quotas.

Here's the flow:

- The student writes code in the **in-browser code editor**, which supports multiple languages
- On submission, the **Exam Service forwards the code to Judge0**
- Judge0 spins up a **sandboxed container** for that specific execution — with CPU limits, memory limits, and a time quota
- Results and logs are returned to the Exam Service for scoring and stored as an audit trail

The key principle here is **zero trust execution**. We assume every piece of student code could be malicious. The sandbox ensures that even if it is, it can't escape, can't consume unbounded resources, and can't affect any other student's session.

This is exactly how platforms like LeetCode and HackerRank handle code execution in production."

[Advance slide]

🔪 SLIDE 9 — Cloud Infrastructure Architecture

[Pause for a moment — signal the transition clearly]

"Alright — we've covered the application. Now let's go up a level and talk about the **cloud infrastructure** that powers all of this.

Everything we just talked about — the microservices, the AI proctoring, the code execution — it all needs to run somewhere that is reliable, secure, scalable, and observable.

We chose **Microsoft Azure** as our cloud provider, and **Azure Kubernetes Service** as the orchestration layer. Let me walk you through what we provisioned and why each decision

was made."

[Advance slide]

🔪 SLIDE 10 — Azure Cloud Architecture & Provisioned Resources

"Here's a full inventory of our Azure infrastructure. Let me go through this by category.

Compute & Orchestration — AKS. The heart of our infrastructure is Azure Kubernetes Service. We didn't just create one node pool — we created **dedicated node pools** for application workloads and AI workloads separately. Why? Because AI inference is GPU-intensive and has completely different resource characteristics than a REST API pod. Mixing them on the same nodes would be inefficient and potentially cause interference.

Database — Azure PostgreSQL Flexible Server. We use PostgreSQL for all relational data — student records, exam configurations, audit logs. We enabled **geo-redundant backups**, which means backup copies are stored in a secondary Azure region. If an entire data center goes down, we're not starting from zero.

Storage & Registry. Azure Blob Storage hosts AI models, student documents, and profile images. Azure Container Registry holds all our Docker images — it's our private hub, so we're never pulling from public Docker Hub in production.

Networking & Ingress. We set up a **Virtual Network** so all resources are on a private network. Traffic enters through the **Azure Application Gateway**, which handles SSL termination and WAF. **Private Endpoints** ensure that resources like PostgreSQL and Key Vault are never exposed to the public internet — they're only reachable from within our VNet. **Private DNS Zones** make internal name resolution clean and reliable.

Security & Identity. **Azure Key Vault** stores every secret, certificate, and encryption key. **Microsoft Entra ID Workload Identity** is how our pods authenticate to Azure services — without any passwords or access keys in environment variables. We'll look at this more closely in a moment.

Monitoring & Observability. **Log Analytics Workspace** aggregates all logs. **Application Insights** provides application-level telemetry — request rates, failure rates, response times. We also set up Prometheus and Grafana for Kubernetes-level metrics.

This is not just a development cluster with some services bolted on. This is a production-grade architecture."

[Advance slide]

🔪 SLIDE 11 — (Visual / Diagram Slide)

[Describe the diagram if shown]

"This diagram shows how all those resources connect to each other. Notice that the flow is deliberate — traffic enters only through the Application Gateway. Everything else is on the private network. This is the principle of **defense in depth** — multiple layers of security, so no single point of failure exposes the system."

[Advance slide]

🔪 SLIDE 12 — Traffic Flow & WAF Security

"Let me walk you through exactly what happens when a student's browser makes a request.

Step 1 — HTTPS Client Request. The request hits our **public IP**, which is attached to the Application Gateway v2. This is the only public entry point into our entire infrastructure.

Step 2 — Decryption & Inspection. SSL terminates at the Application Gateway using certificates stored in Azure Key Vault. Then, the **Web Application Firewall** kicks in — running **OWASP 3.2 rules in Prevention mode**. This means it's not just detecting attacks — it's actively blocking them. SQL injection, XSS, path traversal — all filtered out before the request ever reaches a pod.

Step 3 — Forwarding. Clean HTTP traffic is forwarded to our **internal Kubernetes Gateway — powered by Envoy**. Envoy then routes to the appropriate service based on path and subdomain rules. The entire backend — AKS nodes, databases, key vault — is on the private subnet. No direct public exposure whatsoever.

This architecture means that even if someone discovers our application's internal service IPs, they have no network path to reach them from outside."

[Advance slide]

🔪 SLIDE 13 — Infrastructure as Code (Terraform)

"Now, how was all of that infrastructure actually created? Manually clicking through the Azure Portal would be a nightmare to reproduce, maintain, or audit.

Everything is defined as **Infrastructure as Code using Terraform**. Let me highlight the key design choices.

Modular Setup. The Terraform code is split into dedicated modules — networking, AKS, PostgreSQL, Key Vault, storage, and identity. Each module has a single responsibility. If I need to change how networking is set up, I touch the networking module only. Nothing else is at risk.

Remote Backend. Terraform state — which is the record of what's actually deployed — is stored in **Azure Blob Storage with blob leases**. The blob lease acts as a lock, so if two engineers run `terraform apply` simultaneously, the second one is blocked until the first completes. No race conditions, no state corruption.

Strict Parameterization. There are zero hardcoded values anywhere in the code. Everything is a variable — region, resource names, node counts, SKUs. We use `.tfvars` files per environment and strict tagging with `Environment` and `Owner` tags on every resource. This means we can always answer: what is this resource for, and who owns it?

The result is an infrastructure that can be torn down and rebuilt completely from code — in a new subscription, a new region, or for a new client — with a single `terraform apply`."

[Advance slide]

🔪 SLIDES 14, 15, 16 — (Visual / CI-CD / GitOps Slides)

[Describe what's shown — likely the GitOps/CI-CD pipeline]

"This is our **GitOps CI/CD lifecycle**. Every change to the platform — application code, Kubernetes manifests, infrastructure — goes through a structured pipeline.

Developers push code to Git. A CI pipeline runs — building Docker images, running tests, pushing to Azure Container Registry. On merge to main, a CD pipeline triggers — updating the Kubernetes deployment manifests and applying them to the cluster.

The key word here is **GitOps** — Git is the single source of truth. If you want to know what version of the AI service is running in production right now, you look at the Git repository. If you want to roll back, you revert a commit. This makes deployments auditable, repeatable, and reversible."

[Advance slides through 14, 15, 16]

🔪 SLIDE 17 — Microsoft Entra ID Workload Identity (Passwordless Pods)

"This is one of my favourite parts of the infrastructure — and it's something many teams get wrong by taking the easy but dangerous shortcut.

The traditional way to give a pod access to, say, Azure Key Vault or Blob Storage is to create a service principal, generate a client secret, and store it as a Kubernetes secret or environment variable. That secret then lives in your deployment config — and it can leak, expire unexpectedly, or get checked into source control by mistake.

We didn't do that. We used **Microsoft Entra ID Workload Identity**, which enables completely **passwordless pod authentication**.

Here's how it works in four steps:

Step 1 — AKS exposes an OIDC Issuer. AKS acts as an OpenID Connect identity provider for workloads inside the cluster.

Step 2 — Entra ID creates User-Assigned Managed Identities. We created dedicated managed identities for each service — the Auth Service has its own identity, the Exam Service has its own, and so on. This follows the principle of least privilege — each identity only has the permissions it needs.

Step 3 — Federated Credentials bind Kubernetes Service Accounts. Using Terraform, we link each Kubernetes Service Account to its corresponding Managed Identity through Federated Identity Credentials. This is the bridge between the Kubernetes world and the Azure identity world.

Step 4 — Pods authenticate passwordlessly. When a pod needs to access Key Vault or Blob Storage, it exchanges its OIDC token — which Kubernetes issues — for an Azure access token. No secrets, no access keys, no credential rotation headaches.

This is how Microsoft recommends running production workloads on AKS. It's more secure, more maintainable, and frankly, more elegant."

[Advance slide]

🔑 SLIDE 18 — Multi-Tiered Disaster Recovery & Data Resiliency

"Production systems will fail. Hardware dies, regions go down, people accidentally delete things. The question isn't *if* — it's *when*. Our disaster recovery strategy covers three tiers.

Database Protection. PostgreSQL Flexible Server has automated backups with **geo-redundant storage** — meaning backup copies are replicated to a secondary Azure region. If the primary region experiences an outage, we can restore from the secondary copy and be back up with minimal data loss.

Storage Protection. For Azure Blob Storage — where our AI models, student documents, and profile images live — we enabled **Blob Versioning** and **Soft Delete**. Soft delete means if a file is accidentally deleted, it's not gone immediately. It's retained for **7 days**, giving us a recovery window for human error or application bugs.

Security & Key Protection. Azure Key Vault is configured with **Soft Delete and Purge Protection**. Soft delete retains deleted secrets and certificates for **90 days**. Purge protection means that not even an administrator can permanently destroy a key vault resource within that window. This protects us from both accidents and malicious insider actions.

Three tiers. Database, storage, secrets. Each with independent recovery guarantees. This is what a production DR strategy looks like."

[Advance slide]

🔑 SLIDE 19 — Observability: Prometheus, Grafana & Network Routing

"You can't manage what you can't measure. Our observability stack is built on the industry-

standard pairing of **Prometheus and Grafana**.

Metrics Collection. Prometheus scrapes metrics from AKS nodes, the Kubernetes API server, and individual pod metrics. This gives us visibility into CPU usage, memory pressure, pod restarts, request rates — everything we need to understand how the system is behaving under load.

Visualization. Grafana consumes those metrics and presents them through dashboards. We exposed Grafana through a dedicated subdomain — `grafana.clahaanacademy.online` — so the team can access dashboards from anywhere without being inside the cluster.

Routing. To make this work, we configured our Envoy kGateway to route traffic destined for the Grafana subdomain to the monitoring namespace. We also updated the **NetworkPolicy** — called `gateway-netpol` — to explicitly allow outbound traffic from the gateway to the monitoring namespace. This is important because by default, Kubernetes NetworkPolicies deny inter-namespace traffic.

The result is that we can look at a dashboard and know, in real time, whether the system is healthy — and if something goes wrong, we have the data to understand exactly what happened and when."

[Advance slides 20-23 — describe any visuals briefly or transition smoothly]

🎤 SLIDES 20-23 — (Architecture Diagrams / Additional Visuals)

[Walk through any remaining diagrams]

"These slides bring together the full picture — showing how the application services, the Kubernetes layer, the Azure networking, and the observability stack all connect into one cohesive system.

Every component we've discussed talks to the others through well-defined interfaces. Traffic flows through the gateway. Services authenticate via workload identity. Data persists in protected, backed-up storage. Everything is monitored, logged, and traceable."

🎤 SLIDE 24 — Thank You

[Slow down, speak clearly, make eye contact]

"And that brings me to the end.

To summarize what we covered today:

We started with the **why** — the need for a secure, self-hosted, automated exam platform.

We walked through the **application architecture** — six purpose-built microservices, an AI proctoring pipeline, sandboxed code execution, and a fully automated student journey.

Then we went into the **cloud infrastructure** — a production-grade Azure setup with AKS, private networking, WAF security, Infrastructure as Code, passwordless pod authentication, multi-tiered disaster recovery, and full observability.

This project gave me the opportunity to apply not just cloud and DevOps concepts, but also my learnings from the GH200 certification — understanding GPU workloads deeply, which directly informed how we designed and sized the AI inference layer.

I'm happy to take any questions. Thank you."

[Smile, step back slightly, invite questions]

PRESENTER TIPS

- **Time check:** Aim ~1.5-2 mins per slide. Spend the most time on Slides 5, 6, 10, 17.
- **Slow down on technical terms** — give the audience a second to absorb acronyms (JWT, OIDC, WAF, etc.)
- **Use hand gestures** when listing items — "first... second... third..."
- **Pause after transitions** between Application (Slides 1-8) and Cloud (Slides 9-23) sections
- **For Q&A**, anticipate: "Why not use AWS/GCP?", "How does Judge0 scale?", "What happens if the AI service goes down during an exam?"